



دانشکده مهندسی کامپیوتر

سیستم‌های عامل

فاز ۲ پروژه

نیم‌سال اول

سال تحصیلی ۱۴۰۴-۱۴۰۵

در انجام پروژه به نکات زیر توجه نمایید:

- برای تحویل پاسخ این پروژه لازم است در یک فایل zip تمام فایل هایی که تغییر دادید + یک گزارش مختصر از آنچه انجام داده‌اید را قرار بدید و یک فایل به نام OS_P2_student-id1_student-id2.zip در سامانه کورسز ارسال کنید (جزئیات بیشتر در بخش توضیحات پایانی توضیح داده شده است).
- کامنت گذاری در کد جهت افزایش خوانایی انجام شود و هر تکه از کد در بخش مخصوص خود نوشته شود.
- در صورت هر گونه مشکل می‌توانید به اکانت‌های تلگرام زیر پیام دهید:
@Parsa22000 و @lkavehamdl
- در صورت کشف تقلب نمره ۰ برای تمام طرف‌های درگیر در نظر گرفته خواهد شد.
- مهلت انجام این پروژه تا ساعت ۲۳:۵۹ جمعه ۳۰ آذر است.

فهرست مطالب

۳	 بخش اول: CFS
۶	 بخش دوم: COW Fork
۹	 توضیحات پایانی
۹	 توضیحات
۹	 آنچه که باید ارسال کنید

بخش اول: CFS

تا کنون در درس سیستم عامل با الگوریتم های مختلف زمانبندی پردازش ها آشنا شده اید. در سیستم عامل xv6 زمانبند round robin به عنوان تنها زمانبند موجود استفاده می شود. در `scheduler()` فایل `proc.c` می توانید مشاهده کنید که هر بار یک پردازش انتخاب می شود و برای یک کوانتوم زمانی ($1 \text{ tick} \sim 10 \text{ ms}$) اجرا می شود. سپس به هنگام رخ دادن timer interrupt ادامه ی scheduler اجرا می شود. یعنی به هر پردازش به یک اندازه زمان برای اجرا می رسد.

این الگوریتم به هر پردازش اولویت یکسانی برای اجرا می دهد. اما آیا اولویت اجرای یک پردازش CPU intensive و IO intensive با هم برابر است؟ سیستم عامل لینوکس برای حل این مشکل از الگوریتم زمانبند CFS¹ استفاده می کند. این الگوریتم اولویت اجرا را به پردازش های می دهد که کمترین استفاده را از CPU کرده است. پیشنهاد می کنیم برای یادگیری جزئیات این الگوریتم در مورد آن در سطح اینترنت جستجو کنید.

در الگوریتم CFS برای هر پردازش مقداری تحت عنوان virtual runtime تعریف می شود. این مقدار نشان می دهد که پردازش چقدر از CPU به صورت مؤثر استفاده کرده است. در هربار اجرای یک پردازش، $vruntime$ آن به اندازه مضربی از Δt آن پردازش زیاد می شود.

همانگونه که پیش تر گفته شد، در الگوریتم کنونی زمانبند xv6، بعد از مدت 10 ms در هر پردازش، یک وقفه زمانی (timer interrupt) رخ می دهد و ادامه الگوریتم `scheduler()` اجرا می شود. در الگوریتم CFS هر پردازش وزنی دارد و زمانی که به هر پردازش داده می شود به آن وزن وابسته است. این یعنی پردازش با وزن بیش تر در صورتی که CPU را در اختیار بگیرد، می تواند بیش تر از آن استفاده کند.

یکی از ویژگی های منحصر بفرد این الگوریتم، استفاده از ساختمان داده Red Black Tree است. این درخت یک balance tree است و عملیات بر روی آن $O(\log n)$

¹Completely Fair Scheduler

طول می‌کشد. برای سادگی در پیاده‌سازی، لزومی به پیاده‌سازی این ساختمان داده نیست و هر ساختمان داده‌ای که بتواند پردازش‌های را اضافه کند و یا پردازش با کوچکترین `vruntime` را برگرداند مورد قبول است. پیاده‌سازی ساختمان داده گفته شده اختیاری است.

برای اینکه امکان تغییر وزن برای یک پردازش وجود داشته باشد، سیستم‌کالی تحت عنوان `chpnice`^۲ تعریف کنید که با گرفتن `pid` مقدار `Nice Value` آن را تغییر دهد. در لینوکس وزن هر پردازش رابطه زیر را با `Nice Value` دارد:

$$w_p = 1024 \times 1.25^n$$

که w_p وزن پردازش p و n مقدار `Nice Value` آن است. مقدار `Nice Value` از -20 تا +19 است.

در صفحه بعد شبه کدی وجود دارد که الگوریتم CFS را نشان می‌دهد. توجه کنید که این شبه کد صرفاً برای کمک به درک شما می‌باشد و پیاده‌سازی نهایی شما ممکن است تفاوت‌های بسیاری با آن داشته باشد.

Structures

- 1 **Structure CFS_Runqueue (Scheduling Queue):**
 - 2 • `runnable_tree`: `rbtree` (Red-Black Tree, keyed by `vruntime`)
 - 3 • `nr_running`: `int` (Number of runnable entities)
 - 4 • `min_vruntime`: `int` (Minimum `vruntime` in the tree)
 - 5
 - 6 **Structure Sched_Entity (Process):**
 - 7 • `vruntime`: `int` (Virtual Runtime)
 - 8 • `weight`: `int` (Share weight based on nice value)
 - 9 • `state`: `enum` (e.g., `RUNNABLE`, `SLEEPING`)
-

²Change Process Nice Value

Algorithm 1 CFS Scheduler Logic

▷ **Global Parameters**

1 $TargetLatency \leftarrow \dots$ ▷ Time interval for all runnable processes
2 $MinGranularity \leftarrow \dots$ ▷ Minimum slice a process can get

3 **function** **TIMESLICE-FOR**($entity, rq$)
4 $total_weight \leftarrow \sum_{e \in rq.tree} e.weight$
5 $ideal \leftarrow TargetLatency \cdot \left(\frac{entity.weight}{total_weight} \right)$
6 **return** $\max(ideal, MinGranularity)$

7 **function** **ENQUEUE-ENTITY**($rq, entity$)
8 ▷ Ensure monotonicity: new vruntime cannot be far behind
9 **if** $rq.nr_running > 0$ **then**
10 $v_{min} \leftarrow \min(rq.tree).vruntime$
11 **else**
12 $v_{min} \leftarrow 0$
13 $entity.vruntime \leftarrow \max(entity.vruntime, v_{min})$
14 **RB-INSERT**($rq.tree, entity$) ▷ Insert into Red-Black Tree
15 $rq.nr_running \leftarrow rq.nr_running + 1$

16 **function** **PICK-NEXT-ENTITY**(rq)
17 **if** $rq.nr_running = 0$ **then**
18 **return** **IDLE_ENTITY**
19 **return** **LEFTMOST-NODE**($rq.tree$) ▷ Task with smallest vruntime

20 **function** **RESCHEDULE**(rq)
21 $prev \leftarrow current_entity$
22 ▷ Update virtual runtime based on physical execution time
23 $\Delta t_{exec} \leftarrow (t_{now} - t_{last_sched}) \cdot SCALE(prev.weight)$
24 $prev.vruntime \leftarrow prev.vruntime + \Delta t_{exec}$
25 ▷ Put previous task back in tree if it is still runnable
26 **if** $prev.state = \text{RUNNABLE}$ **then**
27 **ENQUEUE-ENTITY**($rq, prev$)
28 $next \leftarrow \text{PICK-NEXT-ENTITY}(rq)$
29 **if** $next \neq \text{IDLE_ENTITY}$ **then**
30 **RB-REMOVE**($rq.tree, next$)
31 $rq.nr_running \leftarrow rq.nr_running - 1$
32 **return** $next$

33 **function** **SCHEDULE-LOOP**(rq)
34 **while** **True** **do**
35 $next \leftarrow \text{PICK-NEXT-ENTITY}(rq)$
36 **if** $next = \text{IDLE_ENTITY}$ **then**
37 **RUN-IDLE**
38 **else**
39 $\tau_{slice} \leftarrow \text{TIMESLICE-FOR}(next, rq)$
40 **CONTEXT-SWITCH-TO**($next, \tau_{slice}$)
41 ▷ After run, execution flows to **RESCHEDULE** (via tick/yield)

بخش دوم: COW Fork

همانطور که در درس به شما گفته شده زمانی که سیستم کال `fork()` صدا زده می‌شود، از اطلاعات پردازش والد یک کپی گرفته می‌شود و به پردازش فرزند داده می‌شود، و این یعنی ما از یک داده دو نسخه فیزیکی داریم. در این بخش می‌خواهیم `fork` ³ COW را پیاده سازی کنیم.

به این شکل می‌خواهیم هنگامی که یک پردازش از فراخوانی سیستمی `fork()` استفاده کرد، ابتدا تمام `page` های مربوط به آن پردازش بین او و فرزندش مشترک باشد تا زمانی که یکی از پردازش ها بخواهد یک عملیات `write` انجام بدهد.

در سیستم عامل `xv6` برای این که متوجه بشویم یک عملیات روی یک `page` مجاز بوده است یا نه، از بیت های `flag` موجود در آن استفاده می‌کنیم. به عنوان مثال اگر یک `page` دارای مقدار ۰ برای بیت `write` داشته باشد و روی آن `page` نوشتن صورت بگیرد، با `usertrap` مواجه می‌شویم که دارای `scause=0xf` است.^۴ برای تشخیص دادن این که یک عملیات `write` مجاز است یا خیر باید یک بیت دیگر به این مجموعه بیت ها اضافه کنید و با استفاده از آن متوجه بشوید که آیا این `usertrap` که مربوط به `page fault on write` است به منظور `COW fork` بوده است یا صرفاً یک نوشتن غیرمجاز بوده است.

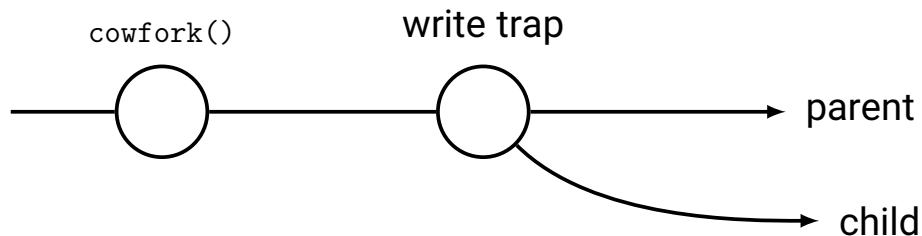
همچنین باید تغییراتی در کد `fork` بدهید که هنگام فراخوانی فضای مجزا و با داده های یکسانی برای فرزند اختصاص ندهد.

برای این بخش شما نیاز دارید که در یک ساختمان داده، برای هر `page` رفرنس‌های به آن `page` را نگه دارید و با استفاده از `spinlock` از آن‌ها مراقبت کنید. پس از اینکه والد یا فرزند تلاش کرد `write` کند و `trap` رخ داد، دیگر داده ها بین

³Copy-on-Write

^۴پیشنهاد می‌کنیم برای آشنایی بیشتر از مکانیزم `paging` در `xv6` فصل `Page Tables` کتاب `xv6` را مطالعه کنید.

این دو پردازه مشترک نبوده و امکان این که از یک نسخه فیزیکی استفاده کنند وجود ندارد. فلذا باید از داده ها یک نسخه فیزیکی دیگر برای پردازه فرزند ایجاد کرد که هر دو پردازه بتوانند داده های خود را نگه دارند.



شکل ۱: page table های والد و فرزند هنگام سیستم کال `cowfork()`

توجه

دستوری که موجب `write trap` شد پس از اینکه `page table` والد و فرزند از یکدیگر جدا شدند باید دوباره اجرا شود.

برای تست کردن این که `cowfork` شما به درستی کار می کند باید یک سیستم کال به نام `physaddr` بنویسید که در سمت کرنل مقدار فیزیکی `PAGE NUMBER` در آن خروجی داده شود.

به عنوان نمونه می توانید از کد صفحه بعد برای تست کردن `cowfork()` استفاده کنید:


```

1  #include "kernel/types.h"
2  #include "kernel/stat.h"
3  #include "user/user.h"
4
5  int main(){
6      char *buf = malloc(4096);
7      buf[0] = 'A';
8      printf("%d", physaddr());
9      if(!cowfork()){
10         printf("Child: initial page number value = %d\n", physaddr());
11         printf("Child: initial buffer val = %c\n", buf[0]);
12         buf[0] = 'C';
13         printf("Child: page number after writing = %d\n", physaddr());
14         printf("Child: buffer value after write: %c\n", buf[0]);
15         exit(0);
16     }
17     wait(0);
18     printf("Parent: after child write, page number value = %d\n", physaddr());
19     printf("Parent: after child write, buffer value = %c\n", buf[0]);
20     return 0;
21 }

```

در این کد ۳ مقدار برای آدرس فیزیکی چاپ می‌شود و در صورتی که مقادیر اول و سوم باهم برابر بودند و با مقدار دوم تفاوت داشتند، COW به درستی پیاده شده است.

توضیحات پایانی

از آنجایی که این فاز باید (ترجیحاً) به صورت گروهی انجام شود، یکی از شما باید فرد دیگر را در پروژه XV6 گیتهاب خود اضافه کند و این فاز و فاز بعدی را در ادامه آن بنویسید. توجه داشته باشید که کامیت‌های هردونفر بررسی می‌شوند و اگر اکثر کامیت‌ها مال یکی از افراد گروه باشد، فرد دیگر نمره‌ای دریافت نخواهد کرد. آیدی گیت هاب ما:

کاوه: lkavehamdl

پارسا: Parsa2201

توضیحات

- این فاز پیش‌نیاز قطعی فاز بعدی است و انجام ندادن آن باعث می‌شود که نتوانید فاز سوم را شروع کنید.
- برای این فاز و فاز بعد نیاز به تشکیل گروه دارید که ثابت بوده و قابل تغییر نیست و در صورت پیدا نکردن همگروهی مناسب و درخواست انجام تکی این پروژه به یکی از دو تدریسار مربوطه اطلاع دهید.
- پروژه شما تحویل آنلاین خواهد داشت بنابراین از استفاده از کدهای دیگران یا کدهای موجود در وب که قادر به توضیح دادن عملکرد آنها نیستید، پرهیزید.
- ابهامات خود را در گروه درس در تلگرام مطرح کنید و ما در سریع‌ترین زمان ممکن به آنها پاسخ خواهیم داد.

آنچه که باید ارسال کنید

- یک فایل zip با نام OS_P2_student-id1_student-id2.zip (student-id را با شماره دانشجویی خود جایگزین کنید) که شامل دو مورد زیر است:
- گزارش خیلی مختصر از آنچه که انجام داده‌اید.
 - پوشه‌ای که در آن کدهای شما وجود دارد. دقت کنید که تنها و تنها فایل‌هایی را که تغییر داده‌اید یا اضافه کرده‌اید را برای ما بفرستید.